



Distributed Systems

Time in Distributed Systems Global State



Recap



Cristian's Algorithm

Berkeley Algorithm

NTP



NTP : Network Time Protocol

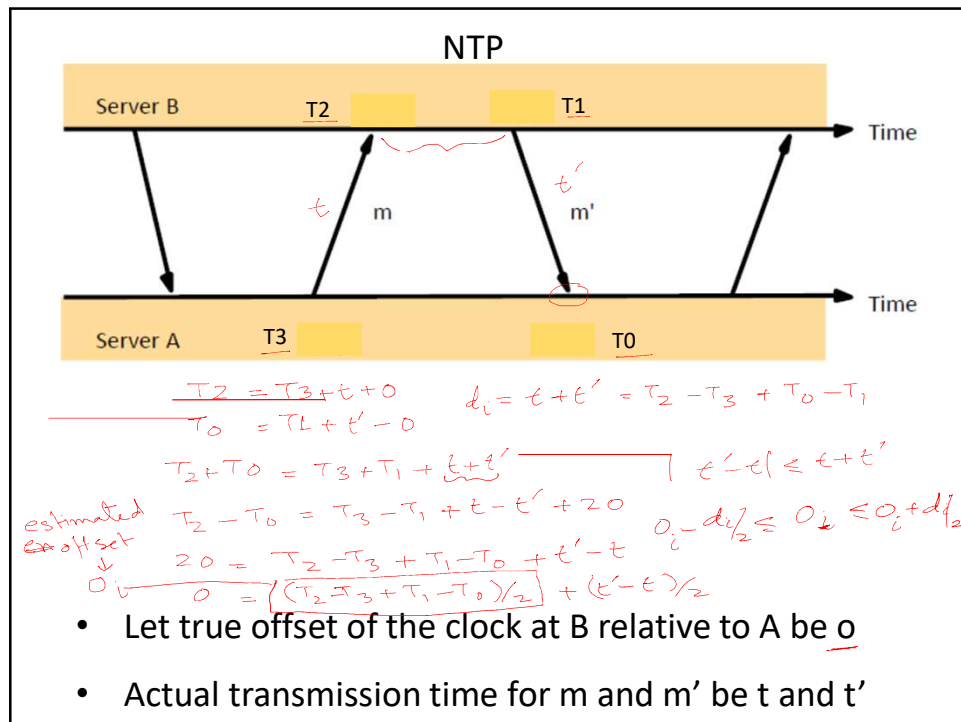
- Achieves accuracy in the range 1-10 msec
- NTP services are provided by a network of servers located across the Internet
- An NTP implementation consists of primary servers, secondary servers and clients
- Hierarchical architecture
 - primary time servers (stratum 1) synchronize to national time standards via radio, satellite etc.
 - secondary servers and clients (stratum 2, 3,...) synchronize to primary servers in a hierarchical manner (stratum 2 servers sync. with stratum 1, stratum 3 with stratum 2 etc.).

Design Aims

- To provide a reliable service that can survive lengthy losses of connectivity
 - ensured by redundant servers and redundant paths
- To enable clients to resynchronize sufficiently frequently
- To provide protection against interference, whether malicious or accidental
 - Uses authentication to check that timing data is from trusted source

- NTP servers synchronize with one another by
 - ^{— broadcast} **Multicast** (usually within high-speed LAN servers) – low accuracies
 - ^{— client-server} **Procedure-call** (higher accuracies than multicast, similar to Cristian's method), or
 - **Symmetric** (usually within multiple geographically close servers and higher level (low strata) servers) – highest accuracies
- Messages are transmitted using UDP / IP

- In procedure-call and symmetric mode processes exchange pair of messages
- Each message bears **timestamps of recent message events**
 - Local times when the previous NTP message between the pair was sent and received
 - Local time when the current message was transmitted
- Recipient notes the local time when it receives the message
- There may be non-negligible processing delay
- Messages may be lost



- For each pair of messages between two servers NTP calculates
 - An offset θ_i (estimate of the actual offset between two clocks) and
 - A delay d_i (total transmission time for the two messages) $\langle \theta_i, d_i \rangle$
- Complex algorithms to combine and filter times $1-10ms$
- But, **just a best-effort service, no guarantees**
- RFC 1305 and 5905 for more details
- SNTP $\rightarrow$

Global State



- A process in a distributed system can obtain a coherent but partial view of the system or a complete but incoherent view of the system
- The global state of a distributed system consists of the local states of the processes, together with the messages that are currently in transit.

Tannenbaum & Steen

IIT ROORKEE ■ ■ ■

Global State Collection

- Many problems in distributed systems can be modelled in terms of detecting global states.
- Checking “stable” properties
 - A stable property is one that persists
 - If y is a **stable property** and y is true at a point in a computation of D , then y is true at all later points in that computation.
 - Examples of stable properties are
 - computation has terminated
 - the system is deadlocked
 - all tokens in a token ring have disappeared.

Applications: Global State



- **Distributed deadlock detection**
 1. **At the decided time** all processes send to some master process the processes or resources for which they are waiting
 2. Master process checks the loop in the resulting graph
 - (Issues with above approach)
- Distributed termination detection
- checkpoint & recovery

IIT ROORKEE

11

4 sites:

R1 stored at S1

R2 stored at S2

P1 runs at S3

P2 runs at S4

Two transactions run concurrently:

P1: lock R1

P2: lock R1

P1: lock R2

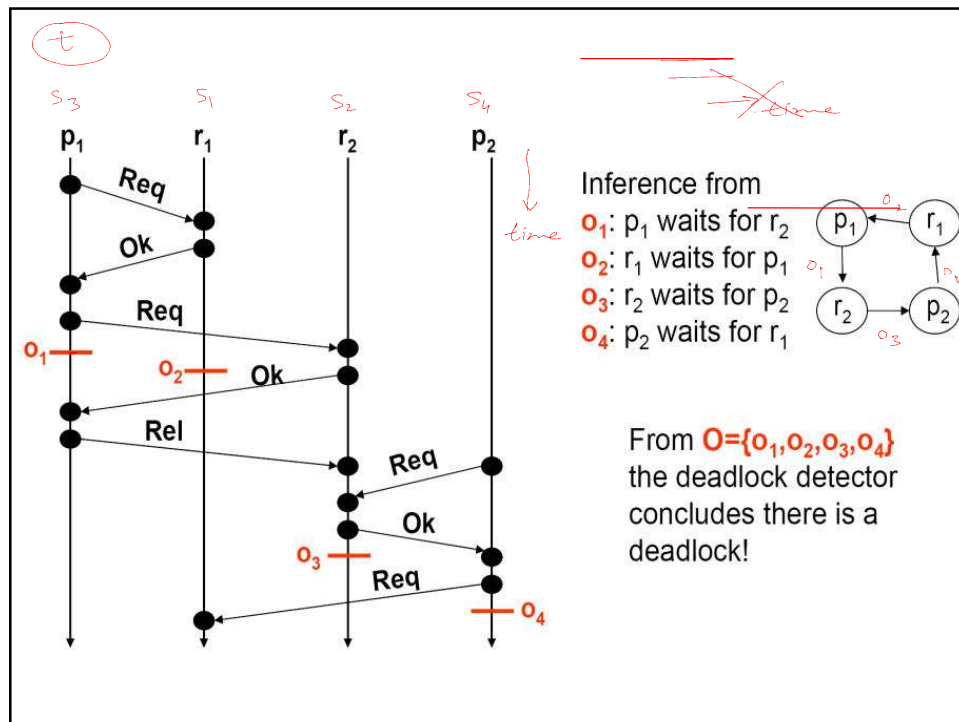
P2: lock R2

P1: unlock R2

P2: unlock R2

P1: unlock R1

P2: unlock R1



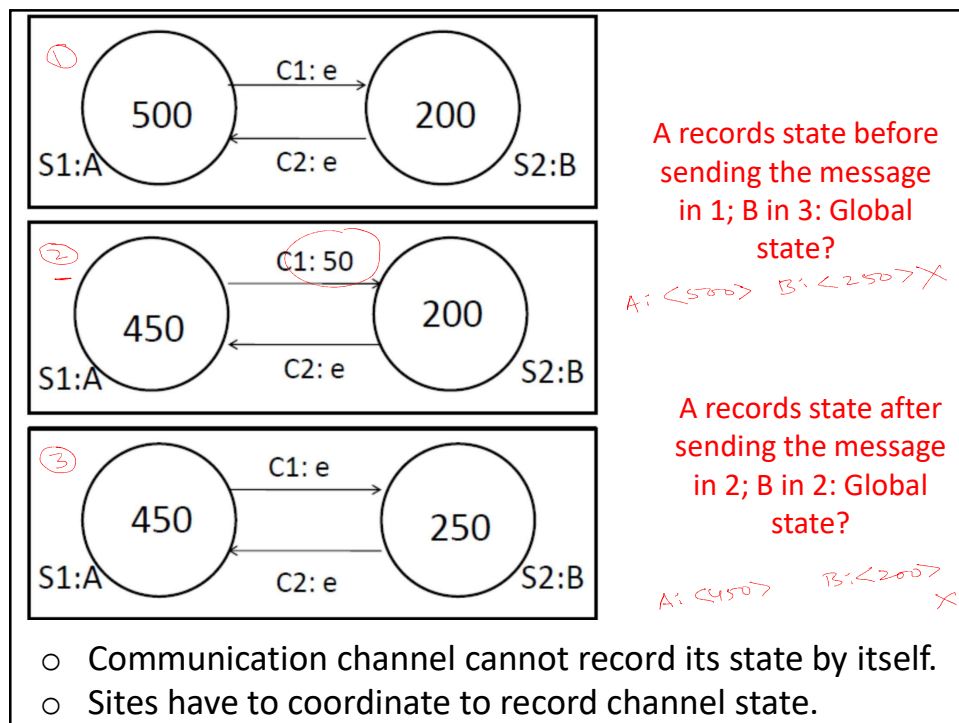
Applications: Global State



Garbage Collector

- Free's up memory which is no longer in use
 - $\Rightarrow$ Check's if a reference to memory still exists (reference counting)
- ✓ Decide a global time when each process checks whether a reference exists to a given object
 - **A references may be in transit**
- ✓ Each process informs which reference it has sent **before the decided global time**
- ✓ Compare them to the references received at decided global time

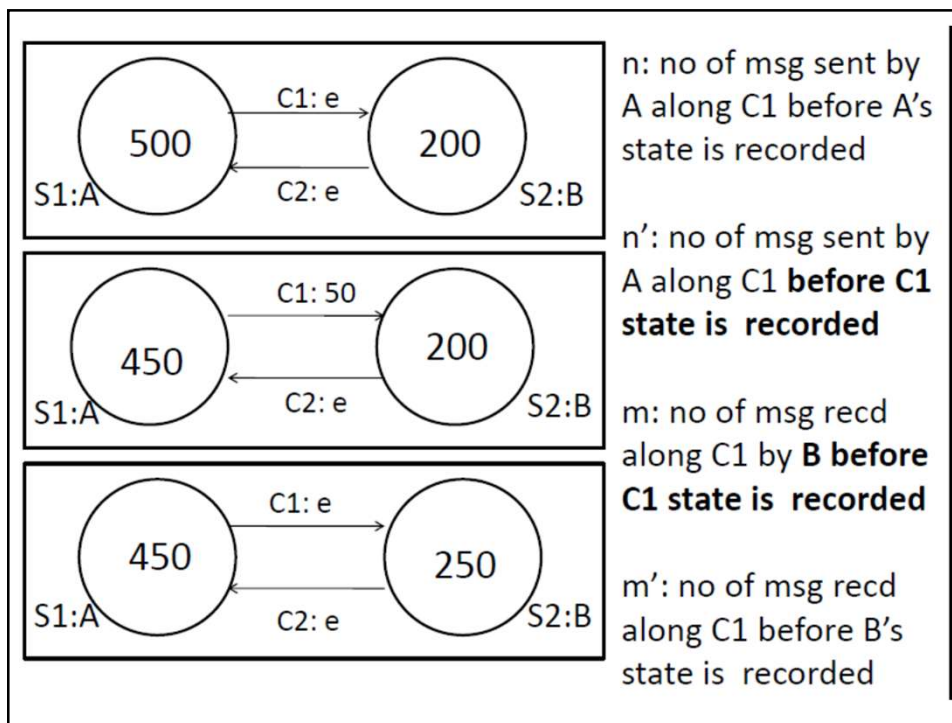
Stable properties may never become true, but once they do they remain true



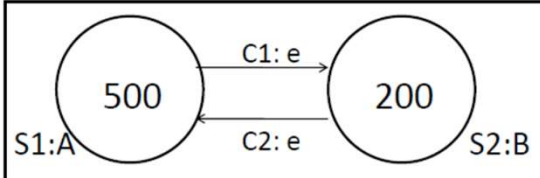
Global State Collection

Issues:

- Need to capture both node and channel states
- system cannot be stopped
- no global clock
- No shared memory



A state in 1; C1, C2 and B in 2: Global state?



n: no of msg sent by A along C1 before A's state is recorded

$$n = 0$$

n': no of msg sent by A along C1 **before C1 state is recorded**

$$n' = 1$$

$$n' > n$$

m: no of msg recd along C1 by **B before C1 state is recorded**

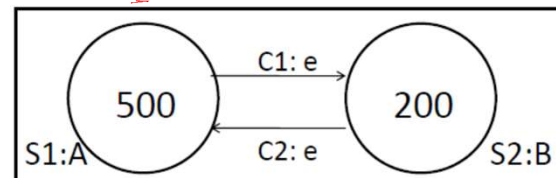
$$m = 0$$

m': no of msg recd along C1 before B's state is recorded

$$m' = 0$$

A: <500> C1: <50> C2: <> B: <200>
X

A state in 2; C1, C2 in 2 and B in 3: Global state?



n: no of msg sent by A along C1 before A's state is recorded

$$n = 1$$

n': no of msg sent by A along C1 **before C1 state is recorded**

$$n' = 1$$

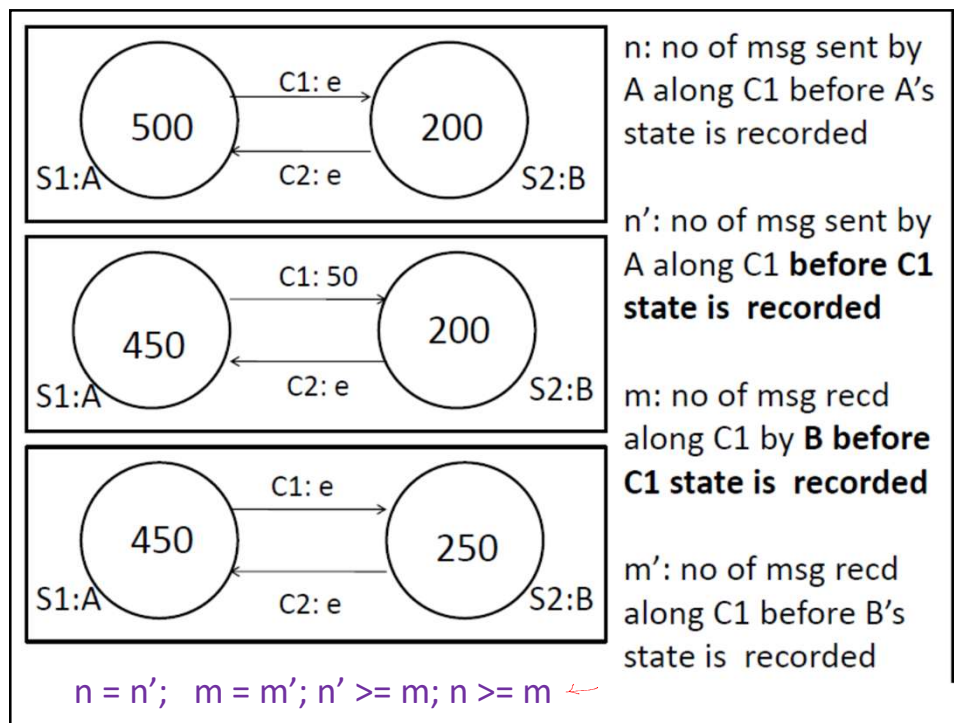
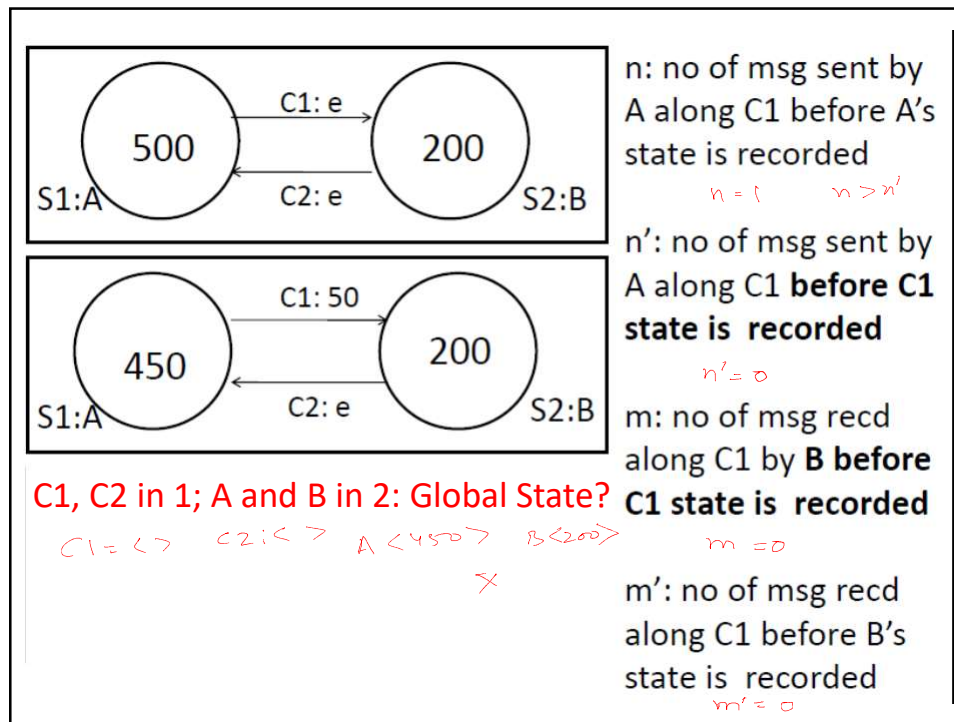
m: no of msg recd along C1 by **B before C1 state is recorded**

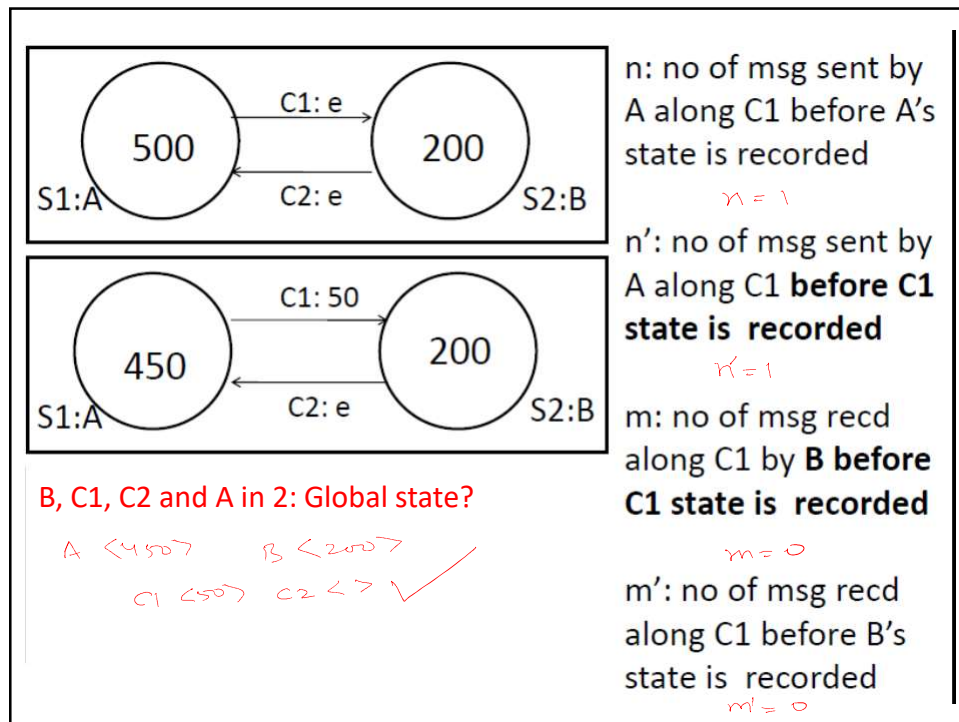
$$m = 0$$

m': no of msg recd along C1 before B's state is recorded

$$m' = 1 \quad m' > m$$

A: <450> C1: <50> C2: <> B: <250>
X





Some notations:

- LS_i : local state of process i
- $\text{send}(m_{ij})$: send event of message m_{ij} from process i to process j
- $\text{rec}(m_{ij})$: receive event of message
- $\text{time}(x)$: time at which state x was recorded
- $\text{time}(\text{send}(m))$: time at which $\text{send}(m)$ occurred

- $\text{send}(m_{ij}) \in \text{LS}_i$ iff

$$\text{time}(\text{send}(m_{ij})) < \text{time}(\text{LS}_i)$$

- $\text{rec}(m_{ij}) \in \text{LS}_j$ iff

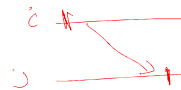
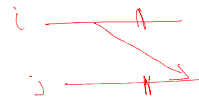
$$\text{time}(\text{rec}(m_{ij})) < \text{time}(\text{LS}_j)$$

- $\text{transit}(\text{LS}_i, \text{LS}_j) = \{ m_{ij} \mid \text{send}(m_{ij}) \in \text{LS}_i \wedge \text{rec}(m_{ij}) \notin \text{LS}_j \}$

- $\text{inconsistent}(\text{LS}_i, \text{LS}_j) =$

$$\{ m_{ij} \mid \text{send}(m_{ij}) \notin \text{LS}_i \wedge \text{rec}(m_{ij}) \in \text{LS}_j \}$$

transit (LS₁, LS₂)
transit (LS₂, LS₃)
transit (LS₁, LS₃)



Global state: collection of local states

$$\text{GS} = \{\text{LS}_1, \text{LS}_2, \dots, \text{LS}_n\}$$

GS is consistent iff

for all $i, j, 1 \leq i, j \leq n,$

$$\text{inconsistent}(\text{LS}_i, \text{LS}_j) = \Phi$$

GS is transitless iff

for all $i, j, 1 \leq i, j \leq n,$

$$\text{transit}(\text{LS}_i, \text{LS}_j) = \Phi$$

GS is strongly consistent if it is consistent and transitless

- corresponds to a consistent global state in which all channels are empty